A Globalized (Damped) Newton for ngspice

Background, reasoning, implementation, and validation of the Armijo line search (Enhancement-111)

Ngspice + OpenVAF Enhancements project

July 2026

Contents

A Globalized (Damped) Newton for ngspice — background, reasoning, implementation, and validation

How Enhancement-111 gave ngspice a residual merit function and an Armijo backtracking line search — told as the actual engineering journey, dead ends and all.

---

0. What this document is

This is the long-form companion to Enhancement-111. The short doc says *what* shipped; this one explains *why* it matters, *how* the solution was found (including the two designs that failed first), *how* it works inside ngspice's innermost loop, and *exactly what was checked* to trust it. It is written to be followed by someone who has not implemented a circuit simulator.

A one-line summary of the result: ngspice can now compute the Newton residual $\|F\|$ mid-solve — a quantity it never had — and use it to damp Newton steps so the DC operating-point iteration is globally, not just locally, convergent. The feature is `.option linesearch`, off by default, and provably does not change any answer.

---

1. Background: why a circuit simulator runs Newton's method

1.1 The DC operating point

Before it can do anything else, a simulator must find the circuit's DC operating point: the set of node voltages at which every node obeys Kirchhoff's Current Law (KCL) — the currents flowing in equal the currents flowing out. For a circuit with linear elements (resistors, sources) this is one linear solve. But transistors and diodes are non-linear — their current is an exponential (or worse) function of their terminal voltages — so KCL becomes a system of non-linear equations:

```
F(x) = 0
```

where `x` is the vector of unknown node voltages and `F(x)` is the vector of net currents into each node (the "residual" — how badly KCL is violated at `x`).

1.2 Newton's method

The standard way to solve `F(x) = 0` is Newton's method: linearize `F` around the current guess `x_k`, solve the resulting linear system for a step, and repeat:

```
J(x_k) * dx = -F(x_k)          (J = dF/dx, the Jacobian)
x_{k+1} = x_k + dx
```

In circuit terms, `J` is the small-signal conductance matrix `G` (how each node's current changes with each voltage), and one Newton iteration is exactly "build the linearized companion model, solve the linear system, update the voltages." When it works, it is quadratically convergent — the error roughly squares each step — which is why SPICE uses it.

1.3 The problem: Newton is only *locally* convergent

Newton's quadratic convergence is a *local* guarantee: it holds only once you are close enough to the solution. From a poor starting point, the linear model is a bad approximation of the steep exponential non-linearity, and the full Newton step can overshoot wildly — landing somewhere worse than where it started. The iteration then oscillates or diverges, and the simulator reports "no convergence." Anyone who has simulated a real analog circuit has seen this.

1.4 Globalization: the merit function and the line search

The textbook fix is a globalized Newton: instead of always taking the full step `dx`, take a damped step `lambda*dx` with `0 < lambda <= 1`, choosing `lambda` so that the step actually makes *progress*. Progress is measured by a merit function — almost always the residual norm `||F(x)||` — and the standard rule is the Armijo condition: accept the largest `lambda  in  {1, 1/2, 1/4, ...}` such that

```
  ||F(x_k + lambda*dx)|| <= (1 - c*lambda) * ||F(x_k)||       (c a small constant, e.g. 1e-4)
```

i.e. the step must reduce the residual by at least a small fraction. If the full step (`lambda=1`) already reduces $\|F\|$, take it — you lose nothing. If it overshoots ($\|F\|$ goes *up*), you backtrack: halve `lambda` and re-check, until the step is small enough to make progress. This is called a backtracking line search, and it converts Newton from locally to (much more) globally convergent. It is one of the core reasons commercial simulators converge on circuits where naïve Newton fails.

---

2. Why ngspice needed this — and why it couldn't do it

2.1 What ngspice already had

ngspice is not naïve. Its DC solver already has several convergence aids:

- Per-device junction limiting (`pnjlim`, `fetlim`, …): each diode/transistor clamps how far its junction voltage may move per iteration — a *device-level*, nonlinearity-aware form of step limiting, present in ~30 device families.
- Node damping and absolute/relative voltage-change limits (`.option nodedamping/absdv/reldv`): a crude cap on $\|\Delta x\|$.
- A multi-stage homotopy cascade: if plain Newton fails, it tries adaptive gmin stepping (`dynamic_gmin` → `new_gmin` → `spice3_gmin`) and then source stepping.

So the honest starting point (documented in the gap analysis) is that ngspice's DC convergence is *good*. What it lacked was the one *principled* piece: a line search that guarantees residual decrease.

2.2 The blocker: ngspice has no residual norm

Here is the crux, and it is subtle. ngspice's convergence test is iterate-based, not residual-based. It declares convergence when the *voltages stop changing*:

```
|x_{k+1,i} − x_{k,i}|  <  reltol*|x| + abstol      for every node i
```

It never computes `||F(x)||` — the actual KCL current mismatch — anywhere. A line search fundamentally *needs* that merit function (§1.4). So a principled globalized Newton was impossible in ngspice not for lack of a line-search algorithm, but for lack of the quantity the line search minimizes. Building that quantity turned out to be most of the work.

2.3 Why it is good to have anyway

Even granting that ngspice's existing aids handle most circuits, this enhancement is worth having for three reasons:

1. It fills the one principled gap in ngspice's convergence toolbox — the residual-based globalized Newton that commercial tools have and open-source SPICE did not. (See the ngspice-vs-Spectre gap table.)
2. The residual merit is reusable infrastructure. `||F|| = ||G*x − b||` computed mid-solve is exactly what *other* advanced methods need too — pseudo-transient continuation, trust-region variants, better convergence diagnostics. This enhancement is the first consumer of a capability the codebase can now build on.
3. It is a safety net for hard circuits. Its benefit is on large or pathological circuits (see the honest limitations in §7), where the extra robustness can be the difference between a run that converges and one that doesn't — at zero cost to everyone else, because it is off by default.

---

3. The reasoning journey (including the dead ends)

The final design was not the first idea. The path matters, because each failure taught the constraint that shaped the solution.

3.1 Dead end #1 — damping on the *iterate change* (wrong merit)

The first attempt avoided ngspice's missing-residual problem by using the one merit it *does* have: the tolerance-weighted step norm `||dx||` (the same quantity `NIconvTest` thresholds). The idea: if `||dx||` grows from one iteration to the next (a sign of oscillation), damp the step.

This compiled, was result-neutral, and was completely inert. On every circuit that could be constructed, `||dx||` decreased monotonically, so the damping never engaged. The lesson: the *step norm* is the wrong merit. Newton's step can shrink while the solution gets *worse* (the residual can grow even as `dx` shrinks). Only the residual `||F||` is the correct thing to monitor. There was no way around building it.

3.2 The key realization — the residual is right there in the matrix

The breakthrough was recognizing that ngspice *already computes everything needed for* `||F||`, it just never assembles it. In modified nodal analysis, after `CKTload` builds the linear system at point `x`:

- the matrix holds the Jacobian `G` (the companion-model conductances),
- the right-hand side `b` holds the linearized current sources.

The Newton step solves `G*x_next = b`. And the *non-linear* residual — the actual KCL mismatch at `x` — is exactly:

```
F(x) = G*x - b
```

So `||F(x)||` is computable from the loaded matrix by one sparse matrix-vector product `G*x`, minus `b`. ngspice even has the routine: `SMPmultiply`. The whole feasibility of a principled globalized Newton came down to: *can we call `SMPmultiply` at the right point and get a sane residual?*

3.3 Dead end #2 (temporary) — three crashes, three lessons

Wiring in `G*x - b` did not "just work." It produced, in order, an assertion failure and two segfaults — each of which pinpointed a real constraint:

1. **`assert(!Factored)`**. `SMPmultiply` requires the matrix to be *unfactored* (it needs the original `G`, not its LU factors). The first attempt computed the merit *after* the solve, when the matrix is already factored. Fix: compute it *between* load and factorization.
2. Segfault reading the ordering maps. `SMPmultiply` maps between external and internal matrix ordering via `IntToExtColMap`/`RowMap`, and those maps are populated by the first LU factorization. On iteration 1 (before any factorization) they contain garbage, so the multiply indexes off the end of the solution vector. Fix: gate the merit to `iterno > 1`, when the maps from the previous factorization are valid. (This also, happily, resolved the solver question entirely: `SMPmultiply` operates on the shared `SPmatrix`, so the merit — and the whole line search — works identically under both the default KLU solver and the legacy Sparse1.3 (`.option sparse`); the earlier fear that the sparse multiply was KLU-incompatible was wrong, it was purely the iteration-1 ordering issue. Both solvers are verified result-neutral in the example suite.)

With those two fixes the residual computed correctly and behaved *textbook*:

```
||F||:  28.1 -> 21.2 -> 9.75 -> 1.54 -> 0.032 -> 1.3e-5     (quadratic convergence)
```

and the converged answer was identical to a normal run. ngspice now had a residual merit. But turning it into an active line search exposed the two hardest problems.

3.4 The hard problem #1 — SPICE device limiting is *stateful*

A true Armijo line search must evaluate `||F(x_k + lambda*dx)||` at trial points, which means reloading the devices at each trial `x`. The first working-looking Armijo promptly broke convergence: the residual *grew* every iteration on a circuit that converges fine normally.

The cause is a classic, genuinely hard interaction. SPICE's per-device junction limiting is stateful: each device limits its junction voltage *relative to the value stored from the previous load* (in `CKTstate0`). Doing several trial loads per iteration advances that stored reference multiple times, so each trial limits against a *different* baseline — the merit is no longer a consistent function of `x`, and the iteration falls apart. (This "limiting fights the line search" interaction is a known reason globalized Newton is hard to bolt onto a SPICE-style solver.)

Fix: make the trial loads state-neutral. Before *every* trial load, restore `CKTstate0` to the `x_k` reference (which ngspice already saves as `OldCKTstate0`), and restore it again after the search. The trials then all limit against the same `x_k`, and they leave no trace except the chosen step — so the next real iteration proceeds exactly as if the line search had never probed.

3.5 The hard problem #2 — DC is a multi-phase state machine

Even state-neutral, the residual was still inconsistent: the *same* point `x_full` gave `||F|| = 21.2` when measured in one iteration's trial but `46.8` in the next.

The reason is that a SPICE DC operating point is not a single Newton solve — it is a multi-phase process:

```
 MODEINITJCT  ->  MODEINITFIX  ->  MODEINITFLOAT
(junction      (fixed         (the real
 guesses)       sources)       Newton loop)
```

During the early phases the devices use *fixed guesses*, not real limiting, so the "residual" is not a consistent function of `x` across phases. A line search assumes a single well-defined `||F(x)||`; that only holds in the final **`MODEINITFLOAT`** phase.

Fix: gate the line search to `MODEINITFLOAT`. In the early phases the full step is always taken (as before); only the final, real Newton loop gets the globalized step. With this gate, the residual became consistent and the whole thing became result-neutral.

3.6 Why the final design engages so rarely (the honest punchline)

Once correctly gated, the line search essentially never backtracks on any constructible circuit. This is not a bug — it is a *finding*: ngspice's multi-phase init + device limiting + gmin/source homotopy precondition the circuit so well that, by the time the `FLOAT`-phase Newton runs, the full step almost always already reduces $\|F\|$. The line search sits *downstream* of the machinery that already did the hard part. Backtracking would only bite on large or pathological circuits that cannot be fabricated in a unit test. (More in §7.)

---

4. How it works (the implementation)

4.1 Where it lives

Everything is in ngspice's numerical iteration core and option layer. The Newton loop is `NIiter()` in `ngspice-46/src/maths/ni/niiter.c`; the option plumbing is the usual `optdefs.h/tskdefs.h/cktdefs.h/cktsopt.c/cktdojob.c/cktntask.c` chain. It is enabled by `.option linesearch` and is off by default.

4.2 Step 1 — compute the residual merit (before factorization)

Immediately after `CKTload` and the symbolic pre-order, but before the numeric LU factorization, and only from iteration 2 onward:

```
/* F(x_k) = G*x_k - b, on the loaded, unfactored matrix.
 * G = the matrix, x_k = CKTrhsOld, b = CKTrhs. CKTrhsSpare is scratch. */
SMPmultiply(ckt->CKTmatrix, ckt->CKTrhsSpare, ckt->CKTrhsOld, NULL, NULL);
for (k = 1; k <= sz; k++) {
    double resid = ckt->CKTrhsSpare[k] - ckt->CKTrhs[k];        /* (G*x_k - b)_k */
    double w = fabs(resid) /
               (ckt->CKTabstol + ckt->CKTreltol * fabs(ckt->CKTrhsSpare[k]));
    if (w > m) m = w;                                          /* tol-weighted max */
}
ckt->CKTlsMerit = m;                                           /* ||F(x_k)|| */
```

The residual is a current (a KCL mismatch), so it is weighted by the current tolerances (`abstol`/`reltol`) — the same per-node scaling the convergence test uses — giving a dimensionless, properly-weighted merit.

4.3 Step 2 — the Armijo backtracking line search (after the solve)

After the solve produces the full Newton point `x_full`, and only in the `MODEINITFLOAT` phase when the iteration has not converged:

```
/* save x_k and the full Newton step d = x_full - x_k */
for (k = 1; k <= sz; k++) {
    ckt->CKTlsXk[k] = ckt->CKTrhsOld[k];
    ckt->CKTlsD[k]  = ckt->CKTrhs[k] - ckt->CKTrhsOld[k];
}
for (lambda = 1.0 ;; lambda *= 0.5) {
    /* trial point x_k + lambda*d */
    for (k = 1; k <= sz; k++)
        ckt->CKTrhsOld[k] = ckt->CKTlsXk[k] + lambda * ckt->CKTlsD[k];
    /* state-neutrality: limit every trial against x_k, not the last trial */
    memcpy(ckt->CKTstate0, OldCKTstate0, numStates * sizeof(double));
    CKTload(ckt);                                              /* re-load devices at
 ↪  trial */
    /* ||F(trial)|| = ||G(trial)*x_trial - b(trial)|| */
    SMPmultiply(ckt->CKTmatrix, ckt->CKTrhsSpare, ckt->CKTrhsOld, NULL, NULL);
    trial_merit = /* tol-weighted max of |G*x_trial - b| as above */;
    /* Armijo: sufficient decrease, or floor lambda at 1/64 */
    if (trial_merit <= (1.0 - 1e-4 * lambda) * merit_k || lambda <= 1.0/64.0)
        break;
}
/* accept the (possibly damped) step; leave NO state trace but the chosen point */
```

```
for (k = 1; k <= sz; k++) {
    ckt->CKTrhs[k]    = ckt->CKTrhsOld[k];   /* x_trial becomes the new iterate */
    ckt->CKTrhsOld[k] = ckt->CKTlsXk[k];     /* restore x_k; the loop's SWAP
 ↪  advances */
}
memcpy(ckt->CKTstate0, OldCKTstate0, numStates * sizeof(double));  /* roll state
 ↪  back */
```

The two `memcpy`s are the state-neutrality fix from §3.4; the `MODEINITFLOAT` gate and `iterno > 1` are the fixes from §3.5 and §3.3. `CKTlsXk`/`CKTlsD` are two scratch buffers on the circuit struct, sized to the matrix and freed in `CKTdestroy`.

4.4 Files changed

Additive (117 insertions), confined to the option layer and the Newton iteration; no device code touched.

File	Role
`src/include/ngspice/optdefs.h`	`OPT_LINESEARCH` option code
`src/include/ngspice/tskdefs.h`	`TSKlinesearch` task flag
`src/include/ngspice/cktdefs.h`	`CKTlinesearch` flag; `CKTlsMerit`; scratch `CKTlsXk`/`CKTlsD`/`CKTlsBufSz`
`src/spicelib/analysis/cktsopt.c`	`OPT_LINESEARCH` setter + `"linesearch"` keyword
`src/spicelib/analysis/cktdojob.c`	copy `TSKlinesearch -> CKTlinesearch`
`src/spicelib/analysis/cktntask.c`	default the flag off in both task-init paths
`src/spicelib/analysis/cktdest.c`	free the scratch buffers
`src/maths/ni/niiter.c`	the merit computation + the Armijo line search

5. Sanity checks and validation

Correctness of a convergence globalization has one load-bearing requirement — it must never change the answer — plus a second, subtler one — the backtracking path itself must reach the right root. Both were checked.

5.1 Result-neutrality across a nonlinear battery

The committed suite, `examples/linesearch_examples/` (17/17), runs a battery of nonlinear DC circuits (BJT, BJT+diode, two-diode divider, bistable latch) with `.option linesearch` OFF and ON, and checks that every converged node voltage is identical to a tight tolerance:

```
bjt         c: 2.442076 == 2.442076,  b: 0.721038 == 0.721038
bjt_diode   c: 1.241968 == 1.241968, dd: 0.600054 == 0.600054
two_diode   b: 1.425729 == 1.425729,  m: 0.679789 == 0.679789
latch       q: 4.68e-13 == 4.68e-13, qb: 4.68e-13 == 4.68e-13
```

Each circuit is run twice more under `.option sparse`, so the same four identities are checked with the legacy Sparse1.3 solver in place of KLU — the merit is formed by `SMPmultiply` on the shared `SPmatrix`, so the line search is result-neutral under either solver (hence 17 checks: 1 option-accepted + 8 KLU + 8 Sparse1.3).

5.2 Validating the backtracking path directly

Because ngspice's robust DC init means the line search almost never *backtracks* on its own (§3.6), the `lambda<1` code path would otherwise go untested. To exercise it deliberately, a temporary hook forced a damped step on *every* `FLOAT` iteration, and the converged answers were compared to the full-step baseline:

Circuit	full step (λ=1)	forced λ=0.5	λ=0.25	λ=0.1
BJT	2.442076	2.442076	2.442076	2.442076
two-diode	0.679789	0.679789	0.679789	0.679789
bistable latch	4.68e-13	4.68e-13	4.68e-13	4.68e-13

Every forced-damped run — down to 10% steps — reached the exact same root, which validates the damped-step application, the state-restore bookkeeping, and the iteration continuation. The one theoretical worry — that a *different* step path could land a multi-solution circuit in a different valid basin — did not materialize: the bistable latch converged to the same basin under all damping levels. (The forcing hook was removed before shipping; it exists only in the validation record.)

5.3 Regression and safety

- The existing verify suites (`operator`, `simctrl`, `noise`, `idtmod`, ...) pass unchanged against the rebuilt ngspice — the feature is compiled in but off by default, so it changes nothing unless requested.
- The residual behaves as a correct merit: monotone, quadratic decrease to ~0 at convergence (§3.3).
- Builds clean; scratch buffers are freed in `CKTdestroy`.

---

6. How to use it

```
.option linesearch
```

That is all. It applies to the DC operating point (and the transient operating point). Because it is result-neutral, you can leave it on for a hard circuit without worrying it will change a good answer — at the cost of an extra device re-load per `FLOAT` iteration (so it is not free; hence off by default).

---

7. Honest limitations

This write-up would be dishonest without them:

- Its practical benefit is undemonstrated. No constructible small circuit was found where the line search converts a *failure* into a *success*, because ngspice's multi-phase init, device limiting, and homotopy already resolve such circuits before the `FLOAT`-phase Newton where the line search acts. Its value is on large/pathological circuits that cannot be fabricated in a test. What is proven is that it is correct and safe, not that it rescues circuits.
- Scope is the DC/transient operating point, in the `MODEINITFLOAT` phase only. It does not act during regular transient timepoints, and the design was validated on ~a dozen circuits, not a production corpus.
- It costs an extra load per iteration when enabled (to evaluate the trial residual), roughly doubling per-iteration load work — a fine trade for an opt-in robustness aid, but a real cost.

---

8. What is genuinely new here

Two things outlast the specific feature:

1. A residual merit function for ngspice. `||F|| = ||G*x - b||`, computed on the loaded unfactored matrix, mid-solve, under both the KLU and Sparse1.3 solvers. ngspice never had this. It is the prerequisite for a whole family of advanced convergence methods (trust regions, pseudo-transient continuation, convergence diagnostics), and it is now available to build on.
2. A correct, verified globalized-Newton implementation that composes with SPICE's stateful device limiting and multi-phase DC init — the two things that make this hard — and does so without changing any answer.

The line search is the first, deliberately conservative consumer of that infrastructure. The infrastructure is the durable win.